2 The auth system emits signals on login events. Capture the `django.contrib.auth.signals.user_login_failed` signal, and print a message displaying the username and path of the login attempt. See https://mng.bz/1GGZ for details on the arguments passed when receiving this signal. (Note that in production, you should use logging rather than `print()`. See appendix B for more details.)

Summary

- Django supports multiuser websites.
- Authentication proves who a user is, while authorization specifies what actions they can perform.
- User data can be stored in a profile object associated with the `User` account object through a `OneToOneField` relationship field.
- Django supports the use of a custom-defined `User` object as an alternative to a profile.
- The Django Admin supports the addition of inline forms to add user profile fields to the `User` object Add and Update screens.
- When placed on a view, the `@login_required` decorator redirects anonymous users to the login page, asking them to authenticate.
- Django provides a view handling user login but requires you to add a template to your site's look and feel to work properly.
- Login for the Django Admin is handled similarly to site login but using separate views. If you're only using the Django Admin and not otherwise providing user accounts, you avoid the need for a custom login page.
- Authorization code is typically view specific. You can add more detailed permission requirements, as needed.
- Django emits signals when it performs certain actions. These signals can be received in a function, allowing you to perform side effects.
- You can use a signal to automatically create a user profile object whenever a User ORM object gets created. Your signal handler traps the `post_save` signal and checks for a newly created `User` object that wasn't created by a fixture.
- Django provides built-in views for the password reset process but requires you to customize the corresponding templates.
- The password reset process prompts the user for their email, sends them a one-time link, displays a reset form, and then a confirmation message.
- Django provides features for sending email and uses different backends for implementation. The `console.EmailBackend` is used during development, printing the email's content to the console instead of sending it.